



## PROJECT GUIDE AND PRESENTATION NOTES

# SireSoft Scan Tool

How it works, how to run it, and where the data lives

Server, port, commands, database, history, edited receipts, advantages and limitations - everything needed to present the project.

|                |                                                                                                                                        |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Server         | <b>lsnet - 10.0.2.2 (reachable on OpenVPN)</b>                                                                                         |
| Address        | <b><a href="http://10.0.2.2:9001">http://10.0.2.2:9001</a></b>                                                                         |
| Project folder | <b><a href="/home/siresoft/siresoft-receiptiq">/home/siresoft/siresoft-receiptiq</a></b>                                               |
| Database       | <b>PostgreSQL - siresoft_receiptiq_db</b>                                                                                              |
| AI model       | <b>Ollama qwen2.5v1:3b, running on the server itself</b>                                                                               |
| Code           | <b><a href="https://github.com/Engr-Mahnoor-code/siresoft-receiptiq">github.com/Engr-Mahnoor-code/siresoft-receiptiq</a> (private)</b> |

SireSoft - internal document - October 2026

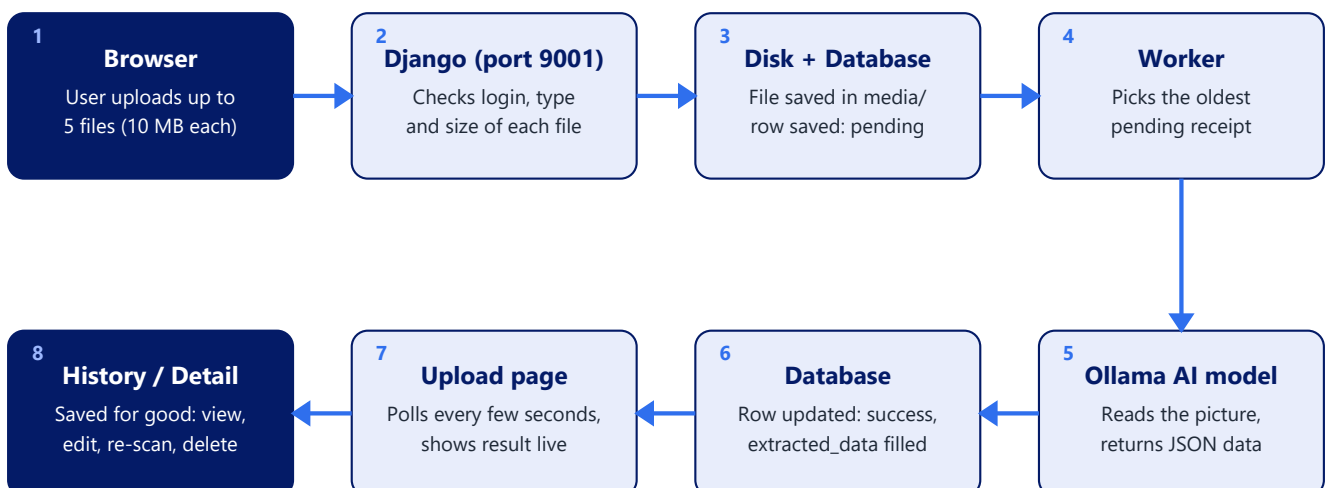
## 01 What the project is

**SireSoft Scan Tool** is an internal web application that turns a photo or PDF of a receipt or invoice into clean, structured data. A user uploads the receipt in the browser; an AI vision model running on SireSoft's own server reads it; and the result - shop name, address, date, every line item, subtotal, tax, other charges, grand total and short insights - is saved in a PostgreSQL database and shown on screen.

Every receipt is kept: the original file, what the AI read, and any correction a person made afterwards, with who made it and when. Nothing is sent to an outside AI service - the model runs on the company server itself.

| Layer             | Technology                                      | What it does                                                         |
|-------------------|-------------------------------------------------|----------------------------------------------------------------------|
| Front end         | HTML, CSS, JavaScript (Django templates)        | Login, Upload, History, Receipt detail, Edit pages in the browser    |
| Back end          | Python 3 + Django 6 + Django REST Framework     | Pages, upload API, access rules, editing, deleting                   |
| Web server        | Gunicorn (3 workers) under systemd              | Serves the app on port <b>9001</b> , restarts it if it ever crashes  |
| AI reader         | Ollama + <code>qwen2.5v1:3b</code> vision model | Reads the receipt image and returns JSON. Runs locally on the server |
| Background worker | Django command <code>process_receipts</code>    | Takes queued receipts one by one and sends them to the AI            |
| Database          | PostgreSQL - <code>siresoft_receiptiq_db</code> | Users and receipts (table <code>receipts_receipt</code> )            |
| File storage      | Server disk - <code>media/receipts/</code>      | Original uploads and the corrected-copy pictures                     |

## 02 How one receipt travels through the system



- 1. Upload.** The user chooses up to 5 files (JPG, PNG, WEBP, GIF or PDF, max 10 MB each) and presses Analyze.
- 2. Saved immediately.** Django stores each file on disk under `media/receipts/` and creates one row in `receipts_receipt` with status `pending`. The browser gets an answer at once - it does not wait for the AI.

3. **Worker picks it up.** The background worker (its own systemd service) marks the row **processing** and sends the picture to the local Ollama model. A PDF is first turned into a picture (page 1).
4. **AI reads it.** The model returns JSON: establishment, items, bill summary, other charges and insights. The code cleans the numbers and text.
5. **Result stored.** The row becomes **success**; vendor name, date and grand total are copied into their own columns, and the full reading goes into **extracted\_data** (and a frozen copy into **scanned\_data**).
6. **Shown live.** The upload page asks the server every few seconds and draws each result as soon as it is ready. The user may even close the browser - the work carries on on the server.
7. **History.** The receipt now appears on the History page, where it can be viewed, edited, re-scanned or deleted.

If the AI cannot read a receipt, the row becomes **failed** with a clear message for the user. If Ollama itself is down, the receipt is not lost: it goes back in the queue and is retried automatically (up to 3 times).

### 03 Do I need to run a command to start the project?

**No.** The project is already running on the server all the time. Connect OpenVPN, open **http://10.0.2.2:9001** in the browser, and it works - without logging in to the server and without running anything.

This is because the app is installed as **systemd services** on the server (1snet, 10.0.2.2). systemd starts them when the server boots and restarts them automatically if they ever stop. Closing your laptop, closing SSH or disconnecting the VPN does not stop the app - it lives on the server, not on your computer.

| Service                  | What it is                              | Needed for                          |
|--------------------------|-----------------------------------------|-------------------------------------|
| siresoft-receptiq        | The website (Gunicorn on port 9001)     | Opening any page                    |
| siresoft-receptiq-worker | Background reader                       | Receipts moving from Queued to Done |
| postgresql               | Database server                         | Storing everything                  |
| ollama                   | Local AI model server (localhost:11434) | Reading receipts                    |
| siresoft-receptiq-tunnel | Cloudflare tunnel (optional)            | Access without the VPN              |

#### Which address works when

| Situation                       | Address                                                     | Works?                                                                       |
|---------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------|
| OpenVPN connected               | http://10.0.2.2:9001                                        | <b>Yes</b>                                                                   |
| OpenVPN disconnected            | http://10.0.2.2:9001                                        | No - 10.0.2.2 is a private address that exists only inside the VPN           |
| OpenVPN disconnected            | Cloudflare tunnel address<br>(https://...trycloudflare.com) | Yes, if the tunnel service is running                                        |
| Server switched off / rebooting | any                                                         | No - wait for the server to come back; the services then start by themselves |

The app answers on port 9001 because `APP_PORT=9001` is set in the server's `.env` file (`/home/siresoft/siresoft-receiptiq/.env`). That file holds all settings and passwords and is never uploaded to GitHub.

Do **not** run `python manage.py runserver` on the server. It is a development server; it would refuse anyway, because Gunicorn already holds port 9001. The real app is already running.

## 04 Server commands

All commands are typed in a terminal on your laptop (Git Bash or PowerShell) while the VPN is connected. Lines starting with `#` are comments, not commands. Type `sudo` commands one at a time - `sudo` may ask for the password.

### 1. Log in to the server

```
# from your laptop, VPN connected; type the server password
ssh siresoft@10.0.2.2

# go to the project folder
cd ~/siresoft-receiptiq
```

### 2. Check that everything is running

```
# app + worker: two lines, both should say "active"
systemctl is-active siresoft-receiptiq siresoft-receiptiq-worker

# should answer HTTP/1.1 302 Found
curl -I http://10.0.2.2:9001/

# more detail (press q to leave)
systemctl status siresoft-receiptiq --no-pager
```

Good result: `active` on each line, and `HTTP/1.1 302 Found` from `curl` (302 means the app is sending a logged-out visitor to the login page - that is correct).

### 3. Start, stop, restart

```
# restart app and worker
sudo systemctl restart siresoft-receiptiq siresoft-receiptiq-worker

# stop them (the site goes offline!)
sudo systemctl stop siresoft-receiptiq siresoft-receiptiq-worker

# start them again
sudo systemctl start siresoft-receiptiq siresoft-receiptiq-worker
```

## 4. Watch what is happening (logs)

```
# worker, live - Ctrl+C to stop watching
journalctl -u siresoft-receiptiq-worker -f

# website, last 50 lines
journalctl -u siresoft-receiptiq -n 50 --no-pager

# errors only
journalctl -u siresoft-receiptiq -p err -n 30 --no-pager
```

The worker log is the best thing to show live: every receipt prints Receipt #84 (file.jpg) - extracting... and then Receipt #84 done in 41.2s - Shop name, total 26.06.

## 5. Put new code on the server (after pushing to GitHub)

```
cd ~/siresoft-receiptiq

# take the newest code from GitHub
git pull

# required on this server after every pull
sudo restorecon -v deploy/*.sh

# only if the database structure changed
./venv/bin/python manage.py migrate

# copies new CSS/JS - otherwise old styles stay
./venv/bin/python manage.py collectstatic --noinput

# load the new code
sudo systemctl restart siresoft-receiptiq siresoft-receiptiq-worker
```

## 6. Public address when the VPN is off

```
# current Cloudflare address (changes on every restart)
journalctl -u siresoft-receiptiq-tunnel \
| grep -o 'https://[a-z0-9-]*\.trycloudflare\.com' | tail -1
```

# 05 Where the data is stored

There are two places: the **database** holds the information, the **server's disk** holds the files.

### Database: PostgreSQL, database siresoft\_receiptiq\_db

| Table            | What one row is                                                       |
|------------------|-----------------------------------------------------------------------|
| receipts_receipt | One uploaded receipt - see the columns below                          |
| auth_user        | One user account (username, email, staff/admin flag, hashed password) |
| django_session   | Who is logged in right now                                            |

## Columns of receipts\_receipt

| Column                               | Meaning                                                                                                                                               |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| id                                   | Receipt number - the same number you see in the address, e.g. /receipt/ <b>84</b> /                                                                   |
| user_id                              | Who uploaded it (links to <code>auth_user.id</code> )                                                                                                 |
| vendor_name, date, total_amount      | Shop name, receipt date and grand total - used by the History list                                                                                    |
| file                                 | Path of the original upload, e.g. <code>receipts/bill_01.jpg</code>                                                                                   |
| content_type                         | File type, e.g. <code>image/jpeg</code> or <code>application/pdf</code>                                                                               |
| status                               | <code>pending</code> -> <code>processing</code> -> <code>success</code> or <code>failed</code>                                                        |
| extracted_data                       | The full reading as JSON: establishment, details, items, bill summary, other charges, insights. After an edit, this holds the <b>corrected</b> values |
| scanned_data                         | What the AI originally read. Never changed by an edit - so before and after can be compared                                                           |
| corrected_file                       | Path of the corrected-copy picture, e.g. <code>receipts/corrected/receipt-84-corrected.png</code> (only after an edit)                                |
| edited_at, edited_by_id              | When and by whom the receipt was corrected                                                                                                            |
| created_at, started_at, processed_at | Uploaded, AI started, AI finished                                                                                                                     |
| error_message, attempts              | Why it failed (if it did) and how many retries                                                                                                        |

## Files on disk

| Folder on the server                                        | What is inside                                                                     |
|-------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>~/siresoft-receiptiq/media/receipts/</code>           | Every original upload, exactly as the user sent it. Never altered                  |
| <code>~/siresoft-receiptiq/media/receipts/corrected/</code> | One PNG per edited receipt: a clean SireSoft-styled copy with the corrected values |

Files are not public. They are served only through the app (`/receipt/84/file/`, `/receipt/84/corrected/`), which checks that the viewer is the owner or an admin. Deleting a receipt removes its row **and** both files.

## 06 Checking that receipts are really stored in the database

This is the proof for the presentation: upload a receipt and show it arriving in the database. Three ways, from easiest to most technical.

### Way 1 - Admin panel in the browser (no commands)

1. Open **`http://10.0.2.2:9001/admin/`** and log in with the admin **username** (the normal login page uses the email; `/admin/` uses the username).
2. Click **Receipts**. You see every receipt in the database: id, vendor, date, total, status, user, upload time. Filter by status or user on the right.

3. Click one receipt to see everything stored for it, including the complete extracted JSON and where its file is saved on disk.
4. The admin view is read-only on purpose, so nothing can be changed by accident.

## Way 2 - PostgreSQL directly (psql)

Log in to the server first (section 4), then open the database:

```
# open the database (server password if sudo asks)
sudo -u postgres psql -d siresoft_receiptiq_db

-- one field per line for wide rows (optional)
\x auto
-- leave psql when done
\q
```

Now type any of these queries. Each ends with a semicolon ;.

How many receipts, per status:

```
SELECT status, COUNT(*) FROM receipts_receipt GROUP BY status;
```

The 10 newest receipts:

```
SELECT id, vendor_name, date, total_amount, status, created_at
FROM receipts_receipt
ORDER BY id DESC
LIMIT 10;
```

Who uploaded what:

```
SELECT r.id, u.username, u.email, r.vendor_name, r.total_amount, r.created_at
FROM receipts_receipt r
JOIN auth_user u ON u.id = r.user_id
ORDER BY r.id DESC
LIMIT 20;
```

Receipts that failed, and why:

```
SELECT id, created_at, error_message
FROM receipts_receipt
WHERE status = 'failed'
ORDER BY id DESC;
```

## Way 3 - Django shell (no database password needed)

```
cd ~/siresoft-receiptiq

# count receipts and print the newest five
./venv/bin/python manage.py shell -c "from apps.receipts.models import Receipt as R; \
print(R.objects.count()); \
[print(r.id, r.status, r.vendor_name, r.total_amount) for r in R.objects.all()[:5]]"

# one receipt: status, total and where its file is
./venv/bin/python manage.py shell -c "from apps.receipts.models import Receipt as R; \
r = R.objects.get(pk=84); print(r.status, r.total_amount, r.file.path)"
```

## 07 Looking at one individual receipt

Every receipt has a number (its `id`). It is in the browser address: `http://10.0.2.2:9001/receipt/84/` is receipt **84**. Replace 84 with your number in the commands below.

### In the app

- History -> **View** opens the receipt: establishment, details, items, bill summary (with insights before the grand total) and insights.
- Under it, the picture: the **Original scan** tab shows the file exactly as uploaded; the **Corrected copy** tab appears after an edit.

### In the database

Main columns of receipt 84:

```
SELECT id, vendor_name, date, total_amount, status, content_type,
       file, corrected_file, created_at, processed_at, edited_at
FROM receipts_receipt
WHERE id = 84;
```

Everything the AI read (or the corrected values), nicely formatted:

```
SELECT jsonb_pretty(extracted_data) FROM receipts_receipt WHERE id = 84;
```

Just the line items, as a table:

```
SELECT item->>'name' AS item, item->>'quantity' AS qty,
       item->>'unit_price' AS unit_price, item->>'total' AS total
FROM receipts_receipt, jsonb_array_elements(extracted_data->'items') AS item
WHERE id = 84;
```

### Its files on disk

```
# all original uploads, newest first
ls -lht ~/siresoft-receiptiq/media/receipts/ | head -20

# all corrected-copy pictures
ls -lht ~/siresoft-receiptiq/media/receipts/corrected/

# path of receipt 84's file, from the database
sudo -u postgres psql -d siresoft_receiptiq_db \
  -c "SELECT file, corrected_file FROM receipts_receipt WHERE id = 84;"
```

To bring a file to your laptop for the presentation, run this on your **laptop** (not on the server):

```
# copy receipt 84's corrected picture to the current folder
scp siresoft@10.0.2.2:~/siresoft-receiptiq/media/receipts/corrected/receipt-84-corrected.png .

# copy one original upload (use the name from the file column)
scp siresoft@10.0.2.2:~/siresoft-receiptiq/media/receipts/bill_01.jpg .
```

## 08 What happens when a receipt (image or PDF) is edited

Sometimes the AI misreads a value. Any user can correct their own receipts, and admins can correct anyone's, with **Edit Receipt** (or the small Edit button on each card). On Save:

- The **original file is never touched**. It stays on disk and on the Original scan tab as proof of what the vendor printed.
- `scanned_data` keeps what the AI read; `extracted_data` gets the corrected values; `vendor_name`, `date`, `total_amount` are refreshed.
- `edited_at` and `edited_by_id` record when and who - the page shows "Edited by ... on ...".
- A new picture is drawn - the **corrected copy** - a clean receipt in SireSoft style with every corrected value, items, totals and insights, stamped "CORRECTED COPY - edited by ... on ...". It is saved as `media/receipts/corrected/receipt-<id>-corrected.png`.
- Editing again simply replaces the corrected copy with a new one.
- **Re-scan** sends the original picture to the AI again; the fresh reading replaces the data and removes the edits and corrected copy.

### Showing one edited receipt in the database

All edited receipts, newest first, with who edited them:

```
SELECT r.id, r.vendor_name, r.edited_at, u.username AS edited_by, r.corrected_file
FROM receipts_receipt r
LEFT JOIN auth_user u ON u.id = r.edited_by_id
WHERE r.edited_at IS NOT NULL
ORDER BY r.edited_at DESC;
```

Before and after for receipt 84 (AI value vs corrected value):

```
SELECT scanned_data->'establishment'->>'name' AS ai_name,
       extracted_data->'establishment'->>'name' AS corrected_name,
       scanned_data->'bill_summary'->>'grand_total' AS ai_total,
       extracted_data->'bill_summary'->>'grand_total' AS corrected_total
FROM receipts_receipt
WHERE id = 84;
```

So yes - for a single edited receipt, whether it was uploaded as an image or a PDF, you can see the original file, the corrected picture, the AI's first reading and the corrected data, side by side.

## 09 History page and who sees what

|                                               | Normal user             | Admin (staff)                                |
|-----------------------------------------------|-------------------------|----------------------------------------------|
| History lists                                 | Only their own receipts | Everyone's receipts, with the uploader shown |
| View / Edit / Delete                          | Their own receipts      | Any receipt                                  |
| Open someone else's receipt by typing its URL | Gets "not found" (404)  | Allowed                                      |
| Admin panel /admin/                           | No access               | Full read-only view of the database          |

History shows receipts that were read successfully (and ones being re-scanned). Failed receipts are not listed there - they are visible in the admin panel and in the database with their error message.

## 10 A 10-minute demo for the presentation

1. Connect OpenVPN. Open **http://10.0.2.2:9001** - the app opens straight away, no command run. Explain: it runs on the server under systemd, port 9001.
2. Log in. Show the Upload page.
3. In a terminal: `ssh siresoft@10.0.2.2`, then `journalctl -u siresoft-receiptiq-worker -f` to watch the worker live.
4. Upload 2-3 receipts (one photo, one PDF). Show Queued -> Analyzing -> Done in the browser while the worker log prints each `Receipt #N`.
5. Open one result: items, bill summary, insights, original picture.
6. Edit one value (e.g. a total) and save. Show "Edited by ...", the Corrected copy tab, and that the Original scan is unchanged.
7. Press Ctrl+C in the log. Open the database: `sudo -u postgres psql -d siresoft_receiptiq_db`.
8. Run the "10 newest receipts" query - the new receipts are there. Run the one-receipt and before/after queries for the edited one.
9. `ls -lh ~/siresoft-receiptiq/media/receipts/` - the uploaded files are on disk.
10. Open /admin/ -> Receipts to show the same data in the browser. Finish with the advantages on the next page.

## 11 Advantages

| Advantage                         | Why it matters                                                                                                                            |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Data stays in the company         | The AI model runs on SireSoft's own server. Receipts, prices and customer details are never sent to OpenAI, Google or any outside service |
| No per-receipt cost               | No API key and no monthly AI bill - only the server's own electricity. Scanning 10 or 10,000 receipts costs the same                      |
| Saves manual typing               | Shop, date, every item, quantities, prices, taxes, charges and totals are filled in automatically, with short insights                    |
| Many formats                      | JPG, PNG, WEBP, GIF and PDF; up to 5 receipts in one go                                                                                   |
| Never loses an upload             | Files are saved before the AI starts. If the AI is down, receipts wait in the queue and are processed automatically when it is back       |
| Fast for the user                 | Upload returns instantly; results appear one by one. The browser can be closed - the server keeps working                                 |
| Human correction with audit trail | Mistakes can be fixed; who edited and when is recorded; the original reading is kept for comparison                                       |
| Original is evidence              | The uploaded file is never altered. A separate, clearly labelled corrected copy is drawn                                                  |
| Secure access                     | Login required; users see only their own receipts; admins see all; files are served only after an access check; passwords stored hashed   |

| Advantage          | Why it matters                                                                                                     |
|--------------------|--------------------------------------------------------------------------------------------------------------------|
| Always on          | systemd starts the app at boot and restarts it on any crash. No one needs to "run" it                              |
| Professional stack | Django, PostgreSQL, Gunicorn, systemd - the same tools used by large production sites. Code is versioned on GitHub |
| Easy to check      | Admin panel, SQL queries and logs show exactly what is stored and what the worker is doing                         |
| Room to grow       | Has a REST API (/api/) - can later connect to accounting software, Excel export, a mobile app or reports           |

## 12 Limitations, and how to handle them

| Limitation                                                                                            | How to handle it                                                                                                         |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Speed depends on the server - on CPU only, one receipt can take from tens of seconds to a few minutes | Results stream in one by one; a GPU or a stronger server makes it much faster                                            |
| The AI can misread faint, crumpled or handwritten receipts                                            | Edit Receipt to correct, or Re-scan. A larger model ( <a href="#">qwen2.5v1:7b</a> ) reads better if the hardware allows |
| Only page 1 of a multi-page PDF is read                                                               | Upload long invoices page by page                                                                                        |
| 10.0.2.2 works only on the VPN                                                                        | Use the Cloudflare tunnel address outside the VPN                                                                        |
| The free Cloudflare address changes after every restart                                               | Re-read it with the journalctl command, or set up a fixed domain                                                         |
| On the VPN address the connection is plain http, not https                                            | Traffic stays inside the VPN; a domain with an SSL certificate can be added later                                        |
| Everything is on one server                                                                           | Take regular database and media backups (command below)                                                                  |

Simple backup (run on the server):

```
cd ~

# database to one file
sudo -u postgres pg_dump siresoft_receiptiq_db > receiptiq-db-$(date +%F).sql

# all receipt files to one archive
tar czf receiptiq-media-$(date +%F).tar.gz siresoft-receiptiq/media
```

## 13 Quick troubleshooting

| What you see                        | What to do                                                                                       |
|-------------------------------------|--------------------------------------------------------------------------------------------------|
| Page does not open                  | Is the VPN connected? Then on the server: <code>systemctl is-active siresoft-receiptiq</code>    |
| Receipt stays on "Queued"           | Worker is not running: <code>sudo systemctl restart siresoft-receiptiq-worker</code>             |
| Receipt "failed"                    | Read the message; check <code>journalctl -u siresoft-receiptiq-worker -n 50</code> ; try Re-scan |
| Changes not visible after an update | Run <code>collectstatic</code> and restart both services; press Ctrl+F5 in the browser           |
| "Bad Request (400)"                 | The address used is not in <code>ALLOWED_HOSTS</code> in <code>.env</code>                       |
| Screen stuck on ":" or "(END)"      | Press <code>q</code>                                                                             |

Full command reference: [docs/COMMANDS.md](#) and [deploy/README.md](#) in the project.